# Setting up your machine (Windows + WSL2)

Do this **before the first lab**. Budget 45 minutes, most of it waiting for downloads. If something fails, write down the exact error message and bring it to the lab — do not spend two hours guessing.

macOS and Linux users: skip to the bottom.

## Why WSL2 and not plain Windows?

This course is about pointers, and the best tool for finding pointer bugs is AddressSanitizer. It tells you *exactly* which line wrote past the end of an array, instead of your program mysteriously crashing ten minutes later.

AddressSanitizer is not available in MinGW/MSYS2 GCC on Windows. WSL2 gives you a real Linux with `g++`, `gdb`, `valgrind` and sanitizers in one command, and it matches the lab machines exactly. You still edit in VS Code on Windows; only the compiler runs on the Linux side.

## Step 1 — Install WSL2

Open **PowerShell as Administrator** and run:

```
wsl --install -d Debian
```

Restart when asked. On first launch Debian will ask you to pick a username and password. **This password is not your Windows password**, and you will need it for `sudo`, so do not lose it.

Check the version:

```
wsl -l -v
```

You want `VERSION 2`. If it says 1, run `wsl --set-version Debian 2`.

## Step 2 — Install the toolchain inside Debian

Open the **Debian** app (not PowerShell) and run:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential gdb cmake git valgrind clang-format
```

Check what you got:

```
g++ --version      # want 11 or newer
cmake --version    # want 3.20 or newer
```

## Step 3 — Install VS Code on Windows

Download from https://code.visualstudio.com/. During installation tick "Add to PATH".

Then install the **WSL** extension: open VS Code, press `Ctrl+Shift+X`, search for `WSL`, install `ms-vscode-remote.remote-wsl`.

## Step 4 — Tell Git who you are

**Never used git before? Read `docs/GIT_BASICS.md` now** — the commands below will make a lot more sense, and you'll need the rest of that tutorial before your first submission regardless.

In the Debian terminal:

```
git config --global user.name  "Your Name"
git config --global user.email "your.email@university.edu"
git config --global init.defaultBranch main
```

## Step 5 — Get the starter project

This course hosts code on **a self-hosted Gitea instance**, not GitHub. The starter project template lives at https://git.pdulab.net/pdu/2440099.

**Important: clone into the Linux filesystem, not into `/mnt/c/...`.** Building on `/mnt/c` is 5–10x slower because every file access crosses between Windows and Linux.

```
cd ~
mkdir -p 2440099 && cd 2440099
git clone https://git.pdulab.net/pdu/2440099 starter
cd starter
code .
```

That last command opens VS Code on Windows, connected to WSL. The bottom-left corner of the window should say **WSL: Debian**. If it does not, you are editing the wrong copy of the files.

VS Code will offer to install the recommended extensions. Say yes.

## Step 6 — Build and run

In the VS Code terminal (`Ctrl+\``, which is already inside WSL):

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug
./build/debug/bin/game
```

You should see all tests pass, then a room with an `@` you can walk around. Press `Q` to quit.

You can also use the CMake Tools buttons in the status bar instead of typing these commands.

## Step 7 — Install Cline, your AI assistant (do not use it yet)

This course's AI assistance runs through **Cline**, a VS Code extension, using **your own DeepSeek API key** — BYOK, "bring your own key." There is no shared department account and no central spending cap; you register with DeepSeek yourself, load a small amount of your own money, and Cline spends against that balance. Budget 15 minutes for this step, most of it waiting for an SMS code and a payment confirmation.

1. **Install Cline.** In VS Code, `Ctrl+Shift+X`, search for **Cline**, install it (`saoudrizwan.claude-dev`). It is not in this repo's recommended extensions from Step 5 — that list is kept short on purpose, so the first thing VS Code asks you to install isn't an AI tool you're not allowed to use for three weeks — so you do need to search for it yourself, right here in Step 7.

2. **Register a DeepSeek account.** Go to https://platform.deepseek.com in a browser (no VPN needed — this is a China-based service). Click **Sign up**.

   - **If you have a Chinese mobile number, use it.** Enter the number, request the SMS verification code, enter the code. This is the fastest path and the one most students in this course will use.
   - If you don't have a Chinese mobile number, sign up with an email address instead — click the "use email" / "邮箱注册" link near the phone field, verify the confirmation code sent to your inbox.
   - Set a real password and store it in a password manager. This account will hold a real (if small) cash balance, so treat it like any other account with money attached.

3. **Add credit to your account.** DeepSeek's API is prepaid: you load a balance, and every request subtracts from it — nothing is billed automatically beyond what you choose to add, and there's no surprise invoice. In the platform's left sidebar, find **Billing** (may show as **充值** / "top up" or your account balance in the top corner) and add funds.

   - **¥20–30 is enough for the entire semester** at this course's expected usage on the model in step 5 below — this course's own cost modeling puts a semester at roughly $1–2 (well under ¥20) per student on this model. You do not need to load more than that, and you can always add more later if you're doing a lot of extra experimentation.
   - Pay with **Alipay or WeChat Pay** if you're in China — both are supported directly on the top-up page. A bank card works too if neither is available to you.
   - If the top-up page shows a different layout than described here (these platforms redesign their billing pages periodically), look for any page whose heading mentions "balance," "余额," "recharge," or "top up."

4. **Create an API key.** Still in the DeepSeek platform, find **API keys** in the left sidebar (sometimes under a "Developer" or "API" section). Click **Create new API key**, give it a name you'll recognize later (e.g. `cline-2440099`), and confirm. **DeepSeek shows you the full key exactly once** — copy it now (there's usually a copy-icon button) before closing that dialog. If you close it without copying, you haven't lost anything important; just delete that key and create a new one.

   - Treat this key exactly like a password: anyone who has it can spend your balance. Paste it directly into Cline in the next step and nowhere else. **Never put your API key in any file inside this repository**, in a chat message, or on any public site — if you ever see a key sitting in a file you're about to commit, stop and remove it before pushing.

5. **Configure Cline with your key, and choose the model.** Open Cline's settings (gear icon in its side panel).

   - **API Provider:** choose **DeepSeek** from the provider dropdown.
   - **API Key:** paste the key you just copied.
   - **Model:** choose **DeepSeek V4.1 Flash** (it may be listed by its model ID, `deepseek-flash`). This is the cheap, fast tier this course's budget is planned around — the larger/reasoning DeepSeek models cost more per

token for no benefit on the kind of question you'll ask this semester, so there's no reason to pick a pricier one.

- If **DeepSeek** doesn't appear as a preset provider (Cline's provider list changes over time), choose **OpenAI Compatible** instead, set the base URL to `https://api.deepseek.com`, paste your key, and type the model name in manually.
- Save. Cline stores the key in VS Code's own secret storage, not in a project file — that's why pasting it here, and nowhere else, is safe.

6. **Verify Cline is actually reading this course's rules — do this once, now, before you rely on it starting week 4.** Cline automatically reads a file called `.clinerules` in the workspace root; it does **not** read `AGENTS.md` directly (`AGENTS.md` is a convention some *other* tools read on their own — this repo keeps both in sync via `scripts/sync_ai_rules.sh`, but Cline specifically only looks at `.clinerules`). To check it's working, open the Cline panel and send exactly one message — this one message is fine even in weeks 1–3, since you're testing the tool, not using it for coursework:

   > What C++ language rules must you follow in this repository?

   A working setup answers with things like "no STL containers," "no smart pointers," "C++14 only" — the actual contents of `AI_RULES.md`. If it gives a generic answer instead:

   - Confirm `.clinerules` exists at the very top of the folder you opened (`ls -a ~/2440099/starter/.clinerules`) — not inside `docs/` or `src/`.
   - Confirm VS Code's workspace root is that folder itself, not its parent — `code .` from inside `starter/` in Step 5 gets this right automatically.
   - Reload the window (`Ctrl+Shift+P` → `Developer: Reload Window`) so Cline re-scans the workspace.
   - Check Cline's own settings for a "Custom Instructions" toggle that might be off.
   - If Cline reports an API error instead of an answer, re-check step 5 — a mistyped key or an empty DeepSeek balance both fail here, not silently.

7. Once step 6 passes, do not send Cline anything else yet. Per this course's staged AI policy (`AI_RULES.md`), weeks 1–3 are AI-off otherwise, checked by closed-book in-lab exercises. Installing and verifying it now just means it's ready when week 4 arrives.

## If something goes wrong

**`wsl --install` says the feature is not enabled.** Virtualization is off in your BIOS, or Hyper-V is disabled. Search your laptop model plus "enable virtualization BIOS".

**`code .` says command not found.** VS Code was installed without "Add to PATH". Reinstall it, or launch VS Code manually and use `Ctrl+Shift+P → WSL: Connect to WSL`.

**The build works but the debugger never stops at a breakpoint.** You are probably running the Release preset. Switch the CMake Tools status bar back to `debug`.

**Arrow keys print `^[[A` instead of moving.** You are running the program through a pipe or through the VS Code Output panel instead of a real terminal. Run it from the integrated terminal.

**Everything is extremely slow.** You cloned into `/mnt/c/Users/...`. Move the project to `~/2440099/` and rebuild.

## macOS

```
xcode-select --install
brew install cmake git clang-format
```

Then steps 4–7 above. Use the **CodeLLDB** extension instead of the GDB launch configuration.

## Linux

```
sudo apt install -y build-essential gdb cmake git valgrind clang-format
```

Then steps 4–7 above.